



80.3K ★

OPEN-SOURCE RAG · V0.25

WHAT IS RAGFLOW?

THE RAG ENGINE FOR REAL DOCS

Your parser is silently destroying answers before your model gets a chance. RAGFlow rebuilds the document layer with DeepDoc, OCR, table-structure recognition, and 11 chunking templates — then bolts on GraphRAG and agents.

DEEPPDOC · OCR · TSR

11 CHUNKING TEMPLATES

GROUNDED VISUAL CITATIONS

GRAPHRAG + AGENTS + MCP

80.3K ★ · 9.2K FORKS



PULKIT TYAGI
Senior Gen-AI Engineer

WHY NAIVE RAG BREAKS

~80% of enterprise data is unstructured. Generic **character-count splitters** hand the model garbage on the exact documents your business actually runs on.



Scanned PDFs — OCR confidence collapses on rotated pages; tables become character soup before they reach the embedder



Tables — spanning cells, merged headers, and projected row headers flatten into unreadable text strings



Multi-column layouts — research papers, decks, and reports lose reading order; paragraphs interleave across columns



Citations — auditors, lawyers, and analysts can't trace an answer back to a specific source region or page



Cross-doc reasoning — naive top-k vector recall misses related-but-reworded facts spread across files



PULKIT TYAGI
Senior Gen-AI Engineer

MEET DEEPDOC

RAGFlow's in-house parser — the piece most frameworks outsource to **PyPDFLoader** and then quietly fail at. Three subsystems run in series on every page.

“

OCR pulls the text. Layout recognition tags 10 region types — Text, Title, Figure, Caption, Table, Header, Footer, Reference, Equation. TSR rebuilds tables cell-by-cell across rows, columns, and spanning cells.

— DeepDoc pipeline

Rotated scans? Confidence-scores OCR at 0°, 90°, 180°, 270° and re-extracts at the best angle. **Alt parsers?** MinerU and Docling are first-class options.



TEMPLATE-BASED CHUNKING

Eleven parsers tuned to document *type*. **Pick the right one** per knowledge base — retrieval quality climbs before you tune anything else.

TEMPLATE	USE CASE
Laws	Preserve clause hierarchy & cross-refs
Paper	Keep abstract → method → results spine
Manual	Section-aware tech docs and SOPs
Book	Chapter & subsection scoped chunks
Presentation	Slide-level chunks with image context
Resume	Field-level extraction (skills, roles, dates)
Q&A	Pair-aware splitting for FAQs & tickets
Table / Picture	Structured rows; image + caption pairs



PULKIT TYAGI
Senior Gen-AI Engineer

RAGFLOW

BY THE SPECS

A serious system on modest hardware. **Docker compose up** and you're running the full stack.

80.3K

GITHUB
STARS

11 / 10

TEMPLATES /
LAYOUT TYPES

4 / 16

CORES / GB
MINIMUM

BYO LLM: GPT-5, Gemini 3 Pro, DeepSeek v4, Claude, Ollama. **BYO store:** Elasticsearch (default) or Infinity. **Connectors:** Confluence, Notion, S3, Google Drive, Discord — synced on a schedule, no glue code.



PULKIT TYAGI
Senior Gen-AI Engineer

RAGFLOW VS FRAMEWORKS

Different optimization targets. **Pick by where your hard part lives** — document quality, or pipeline flexibility.

RAGFLOW

Parser-first, opinionated stack

DeepDoc + TSR ship by default

Visual chunks, grounded UI citations

GraphRAG, agents, MCP built in

Low-code; deploys as one system

LANGCHAIN / LLAMAINDEX

Libraries, assemble components

BYO parser (PyPDF, Unstructured)

Citations & UI are your problem

Agents via add-ons / LangGraph

Max flexibility, much more code



PULKIT TYAGI
Senior Gen-AI Engineer

SELF-HOST IT IN MINUTES

Clone, configure, compose. **Elasticsearch + MinIO + Redis + MySQL** come bundled — one command boots the lot.

● ● ● deploy-ragflow.sh

```
# 1. Clone — requires Docker ≥ 24, Compose ≥ 2.26
git clone https://github.com/infiniflow/ragflow.git
cd ragflow/docker

# 2. Pick a doc store: Elasticsearch (default) or Infinity
#   Set LLM factory + API key in service_conf.yaml.template
#   (GPT-5, Gemini 3 Pro, DeepSeek v4, Claude, Ollama)

# 3. Boot the full stack — ES, MinIO, Redis, MySQL, RAGFlow
docker compose -f docker-compose.yml up -d

# 4. Open the UI, create a knowledge base, upload a PDF
open http://localhost:80
```



**RUN IT ON ONE REAL PDF THIS
WEEK.**



LIKE



COMMENT



SHARE



SAVE